Full-Stack & Mobile Systems at Scale

Real-World Case Study: UrjaGrid — nationwide IoT EV charging platform.
Engineering focus: GitHub collaboration, Android release engineering, security, performance, CI/CD and AI-agent-assisted delivery.

INTERFACE OVERVIEW

Consumer Mobile & Vendor Web Experience

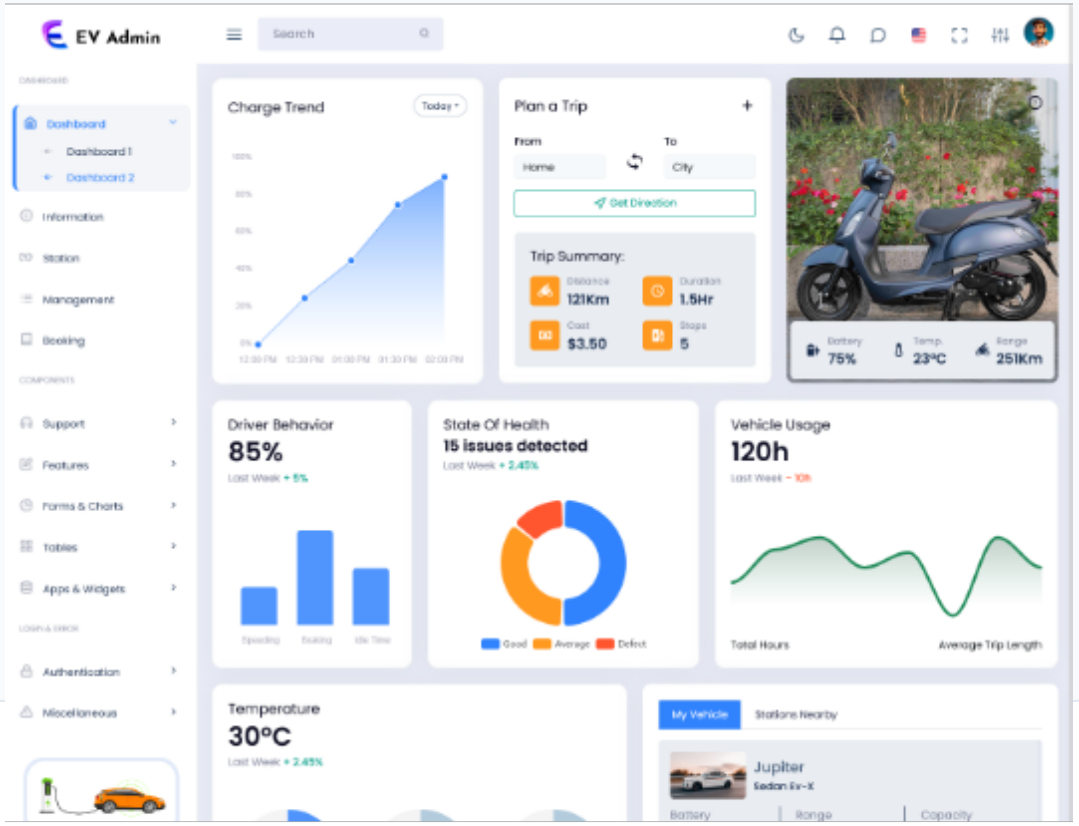
Consumer App (Kotlin / Compose)

Driver View



Vendor Portal (ReactJS / Tailwind)

Operator View



DOMAIN ARCHITECTURE

The UrjaGrid Platform Ecosystem

Vendor React Portal

Station operators register guns, configure tariffs per kWh, and track real-time grid energy draw.

Consumer Kotlin App

Drivers find nearby fast chargers, reserve plugs, process UPI AutoPay, and view battery telemetry.

API Gateway Tier

Nginx reverse proxy routing REST requests to Spring/Node microservices and OCPP WebSockets.

Data & Queue Layer

Redis in-memory status cache, Kafka metering streams, and PostgreSQL ACID storage.

PLATFORM ARCHITECTURE

Microservices Architecture: UrjaGrid

Decompose the platform into independently deployable domain services with clear ownership and communication contracts.

CLIENTS — Kotlin Consumer App • React Vendor Portal • Charger / EVSE Devices

API GATEWAY / EDGE ROUTER • TLS termination, routing, rate limits, request validation

Identity & Access
Login, JWT issuance, roles and permissions

Station & Tariff
Station, connector, pricing configuration

Reservation & Queue
Slot booking, Redis queue allocation

Charging Session
Start/stop lifecycle, OCPP command orchestration

Billing & Payment
UPI mandates, invoices, settlement and refunds

Notification
Push, SMS, email and session alerts

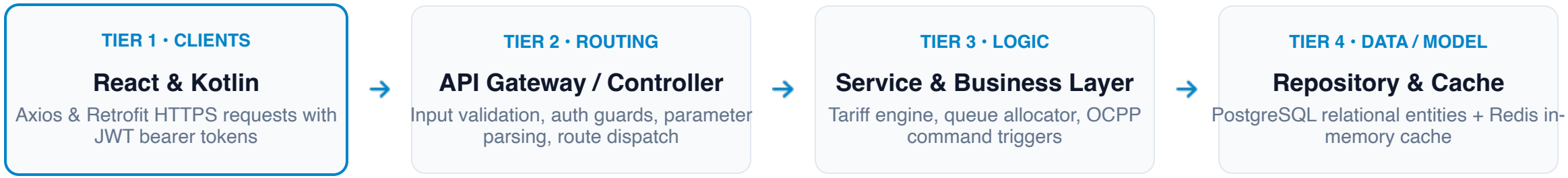
SHARED PLATFORM — Kafka events • Redis cache / queue • PostgreSQL (service-owned schemas) • Observability

Synchronous: REST APIs | Asynchronous: Kafka events | OCPP Gateway maintains charger WebSocket connections

Design principle: each service owns its business capability and data; avoid direct cross-service database access.

SYSTEM ARCHITECTURE

MVC Server Architecture & Request Lifecycle



Separation of Concerns: Controllers never talk to the database directly. They accept DTOs (Data Transfer Objects), delegate to domain services, and return strict JSON HTTP schemas.

INTERFACE CONTRACTS

REST API Payloads: Vendor vs Consumer

Vendor: Register Gun

POST /v1/vendor/guns

Submitted via React dashboard when provisioning physical EVSE hardware.

```
"station_id": "STN_MUM_40001",
"gun_identifier": "GUN_CCS2_A",
"connector_type": "CCS_TYPE_2",
"max_output_kw": 60.0,
"base_tariff_kwh": 18.50,
"ocpp_box_id": "CP_URJA_882"
```

✓ Authenticated via Vendor JWT with DISCOM site permissions

Consumer: Reserve Slot

POST /v1/consumer/reserve

Dispatched from Kotlin app when booking a slot or entering virtual queue.

```
"user_id": "USR_IND_9921",
"gun_id": "GUN_CCS2_A",
"vehicle_reg": "MH01-EA-1234",
"target_soc_pct": 80,
"upi_mandate_id": "MND_UPI_771822",
"eta_minutes": 15
```

✓ Protected by Idempotency Key header to avoid duplicate charges

MODULE 01 • SOURCE CONTROL INTERNALS

Git Storage Trees & Branching Topology

Working Directory

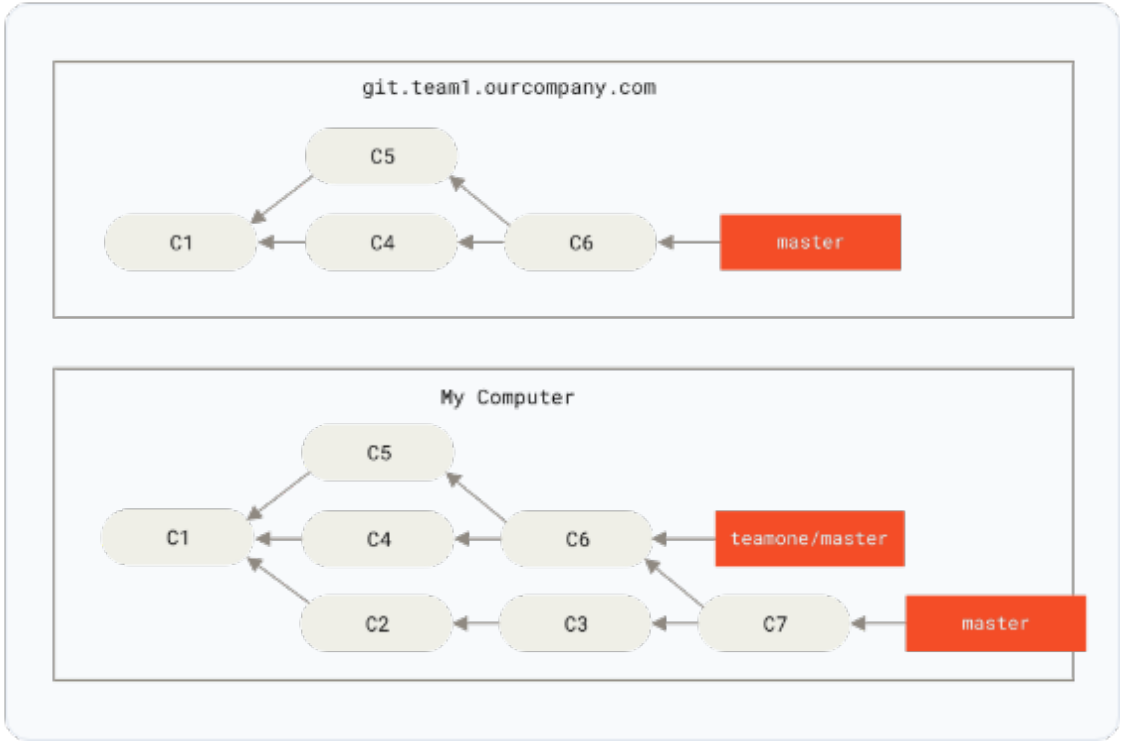
Untracked code workspace where developers write React components and Kotlin coroutines.

Staging Index

Binary tree snapshot prepared via git add before recording a commit.

Commit Object DAG

Directed Acyclic Graph of SHA-1 hashes preserving tamper-proof revision history.



MODULE 01 • PRACTICAL ENGINEERING

GitHub Workflow: Pull, Stage, Push & Merge

A disciplined workflow keeps changes reviewable, collaboration predictable and releases traceable.

1. Sync your branch

git fetch origin

Download remote updates without changing your working files.

git pull --rebase origin main

Reapply your work on the latest main branch to keep history linear.

2. Stage & publish

git add -p

Select only the intended diff chunks.

git commit -m "feat: add upi"

git push -u origin feat/upi

3. Review & merge

Open a GitHub pull request.

Require CI checks and code-owner review.

Merge, squash or rebase according to repository policy. Keep the PR linked to its issue for traceability.

Team rule: protect main, avoid committing secrets, keep PRs small, and use a tagged commit as the source of every production release.

MODULE 01 • PRODUCTION VERSIONING

GitHub Collaboration: From Branch to Production

Extend the existing Git fundamentals into a repeatable team workflow.

1. Branch strategy

main: protected, releasable code

feature/<ticket>-<name>: isolated work

release/<version>: stabilization

hotfix/<incident>: urgent production patch

2. Pull request controls

Small, focused PRs with linked issues

Required reviewers + CODEOWNERS

CI checks must pass before merge

Resolve conversations; squash or merge by policy

3. Repository protection

Block direct pushes to main

Require signed commits where appropriate

Enforce status checks and linear history

Use least-privilege team and token permissions

Example GitHub flow

Create branch → commit → push → open PR → review → CI validation → merge → tag release

Operational practices

Use CODEOWNERS for Kotlin, React and infrastructure paths. Keep secrets out of Git. Prefer short-lived branches and reproducible commit history.

Security: Protect Data, Identity and Network Traffic

Apply defense in depth across the React portal, Kotlin app, APIs and platform services.

Data at rest

Encrypt databases, backups and object storage using managed keys (e.g. AES-256 where supported).

Rotate keys and restrict key access.

Classify PII, payment references and telemetry.

Data in transit

Enforce HTTPS with TLS 1.2+ (prefer TLS 1.3 where supported).

Validate certificates; disable cleartext traffic.

Use secure WebSocket (WSS) for OCPP connections.

Identity & API security

OAuth 2.0 / OIDC or short-lived JWTs.

Role-based access and least privilege.

Validate input; rate-limit sensitive endpoints.

Use idempotency keys for payment operations.

Secrets & supply chain

Store credentials in a secret manager; never commit API keys. Scan dependencies and container images. Pin and review build dependencies; generate an SBOM where required.

Detection & response

Centralize audit logs, alert on unusual access, test backups and incident playbooks. Redact tokens and personal data from logs.

MODULE 02 • IDENTITY & SECURITY

Token-Based Authentication: Tools & Working

A reference flow for securing the React portal, Kotlin app, APIs and internal services.

1 LOGIN

User signs in with credentials / MFA

→

2 ISSUE TOKENS

Identity provider verifies identity and signs JWT

→

3 SEND TOKEN

Client sends access token in Authorization: Bearer

→

4 VALIDATE

Gateway / Spring Security checks signature and claims

→

5 AUTHORIZE

Roles and scopes decide which API action is allowed

ACCESS TOKEN (SHORT-LIVED JWT)

Signed claims: subject, issuer, audience, expiry, roles / scopes. The API validates the signature and claims; it does not trust client-supplied role values.

REFRESH TOKEN (ROTATED)

Used to obtain a new access token without asking the user to log in again. Store securely, rotate on use, revoke on logout or suspected compromise.

TOOLS / CONTROLS **Spring Security + OAuth 2.0 / OpenID Connect • Keycloak / Auth0 (IdP) • JWT + JWKS • BCrypt / Argon2**

HTTPS/TLS • secure mobile token storage • RBAC / scopes • short expiry • refresh-token rotation • audit logs

Authentication confirms who the caller is; authorization checks what that caller is allowed to do.

Performance Engineering: Latency, Memory and Reliability

Extend caching and database indexing with client-side and service-level performance controls.

Android memory management

Profile allocations and heap with Android Studio.

Avoid Activity / Context leaks; clear lifecycle-bound references.

Use pagination and bounded caches.

Decode images at display size; release large bitmaps.

UI & network efficiency

Use Kotlin coroutines and structured concurrency.

Avoid main-thread I/O; render lists with lazy components.

Batch requests where practical; compress payloads.

Use timeouts, retries with backoff and cancellation.

Backend performance

Measure p50/p95/p99 latency and throughput.

Tune connection pools and thread pools.

Use Redis TTLs and cache invalidation deliberately.

Index high-selectivity query paths; avoid N+1 queries.

Observability & budgets

Track cold-start time, ANR rate, crash-free users, API latency, DB query time and memory pressure. Define service-level objectives and regression thresholds.

Performance validation

Run load tests before release; profile real devices; compare baseline vs candidate build. Gate rollout on error rate, latency and crash/ANR health.

MODULE 03 • PERFORMANCE OPTIMIZATION

Multi-Tier Caching & Database Indexing

1. Edge CDN Caching

Cloudflare / CloudFront caches React bundle assets, images, and static station metadata close to users.

Latency: ~15ms

2. Redis In-Memory Cache

Cache-Aside Pattern stores gun statuses (AVAILABLE, CHARGING) with 30s TTL. Avoids pounding DB on repeated polls.

Latency: ~1ms

3. B-Tree DB Indexing

Composite index on (city_id, connector_type, is_active) turns full table scans (O(N)) into binary search (O(log N)).

Query speed: 220ms → 4ms

CONCURRENCY & REAL-TIME QUEUING

Smart Queue System: When All Guns Are Occupied

Redis Sorted Set (ZSET) Queue

When all station guns are in state OCCUPIED, requests are queued in Redis using timestamp epoch as the sort score:

```
// 1. Add driver to station sorted queue
ZADD q:MUM_01:CCS2 1730198400 USR_9921

// 2. Fetch driver rank (0-indexed)
ZRANK q:MUM_01:CCS2 USR_9921 -> Pos #2
```

When an active vehicle unplugs, an OCPP webhook executes ZPOPMIN and alerts the next driver with a 5-minute reservation hold.

Dynamic Wait-Time Calculation

Wait time shown in the Kotlin app is continuously computed from live hardware telemetry:

T_wait = [(TargetSoC - CurrentSoC) × Cap_kWh / Output_kW] + (Rank × Turnaround_Mins)

- **CurrentSoC:** Live vehicle battery % from OCPP MeterValues
- **Output_kW:** Active delivery speed adjusted for grid load
- **Turnaround:** 5-minute buffer for parking & cable reconnection

SYSTEM INTEGRATION • CHARGER COMMUNICATION

OCPP over WebSockets: Real-Time Charger Communication

Open Charge Point Protocol connects charging stations to the Central System (CSMS) through a persistent, bidirectional connection.

1. EV Charger / EVSE

Opens an outbound WebSocket connection to UrjaGrid's CSMS endpoint.

⇄

2. OCPP Gateway

Authenticates the charger, maintains the session and routes OCPP messages.

⇄

3. CSMS Services

Processes status, authorization, charging transactions and remote commands.

⇄

4. Platform Events

Publishes telemetry to Kafka; updates Redis and user-facing app views.

Common OCPP 1.6J messages

BootNotification — charger announces startup

Heartbeat — keeps session alive

StatusNotification — connector state changes

Authorize — validates driver / token

UrjaGrid message flow

Charger sends MeterValues and StatusNotification over the open WebSocket.

UrjaGrid updates live charger state, streams telemetry through Kafka and refreshes availability in the Kotlin app.

Remote operations & security

CSMS can send supported commands such as RemoteStartTransaction and RemoteStopTransaction.

Use WSS (WebSocket over TLS), charger authentication, authorization checks, connection monitoring and message

Key idea: REST APIs serve app requests; OCPP WebSockets provide the persistent, two-way channel between chargers and the charging platform.

MODULE 03 • HIGH-THROUGHPUT INGESTION

Apache Kafka: Producer, Topic Payloads & Consumers

Producer & Payload

5,000+ msgs/sec

Producer: OCPP WebSocket Gateway translates charger pings into serialized events on topic ev.charger.telemetry:

```
"charger_id": "CP_MUM_8812",
"gun_id": "GUN_CCS2_A",
"current_soc_pct": 68.5,
"active_power_kw": 58.4,
"cumulative_kwh": 31.2,
"gun_temp_celsius": 42.1
```

Partition Key: charger_id guarantees sequential delivery per gun within the broker partition.

Independent Consumer Groups

Consumer A: Live Driver Telemetry (Kotlin)

Pipes current_soc_pct into Redis and dispatches SSE frames to the mobile app for real-time battery gauge animations.

Consumer B: Dynamic Billing Engine

Aggregates cumulative_kwh, applies time-of-day tariffs, and triggers auto-stop when UPI mandate spend caps are reached.

Consumer C: Thermal Safety Monitor

Checks gun_temp_celsius. If temp exceeds 65°C, issues an immediate OCPP emergency shutdown command.

MODULE 03 • NON-USER INTERACTION JOBS

Background Schedulers & Bulk Asynchronous Processing

DAILY @ 02:00 AM IST

1. Financial & UPI Settlement

Why asynchronous? Processing 100,000 banking settlements synchronously would time out HTTP request threads and lock database rows.

- Batch charges pending UPI AutoPay mandates.
- Calculates DISCOM power costs vs vendor commission.
- Generates PDF tax invoices stored in S3.

HOURLY & WEEKLY BATCHES

2. Telemetry Rollups & Cold Archival

Why asynchronous? Ingesting 5,000 events/sec bloats PostgreSQL tables to gigabytes daily, degrading query index lookups.

- Downsamples raw 5-sec pings into 1-hour avg metrics.
- Bulk dumps records older than 30 days to AWS S3 / Parquet.
- Keeps primary DB lean for sub-10ms operational queries.

EVERY 15 MINUTES

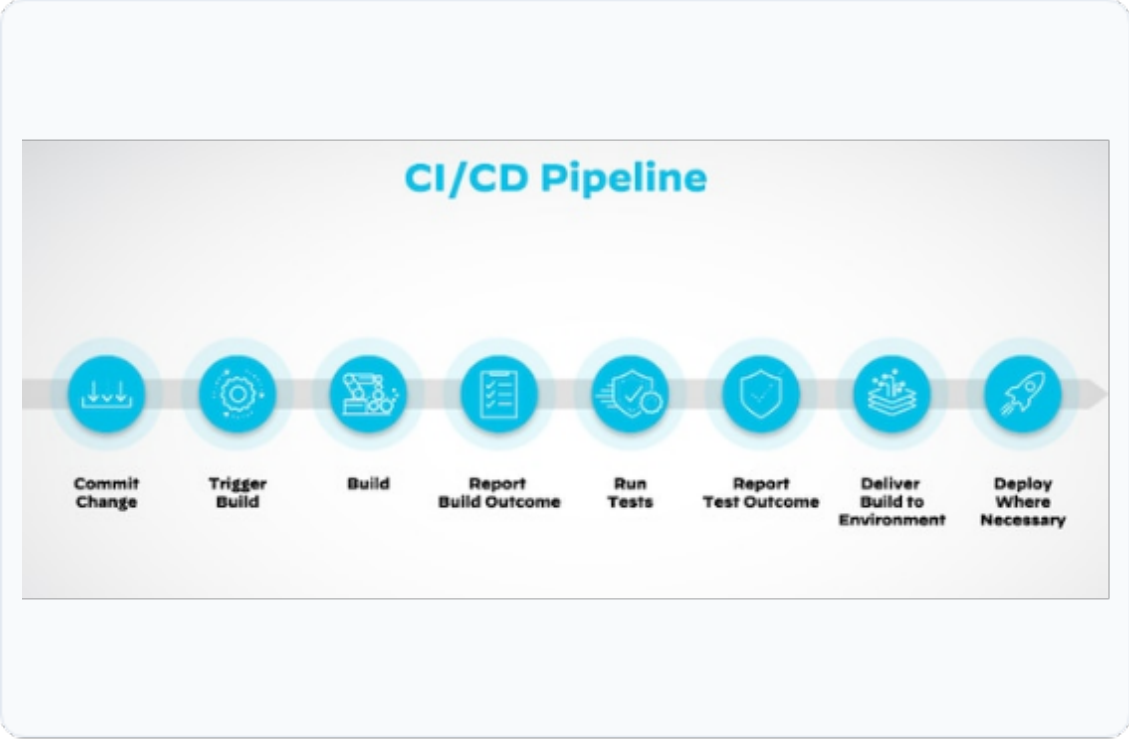
3. Hardware Predictive Maintenance

Why asynchronous? Scanning multi-station voltage fluctuations and connector wear requires heavy analytical calculations.

- Scans for guns with repeated transaction aborts.
- Auto-flags faulty hardware to MAINTENANCE_REQUIRED in Redis.
- Creates technician repair tickets before users arrive.

MODULE 04 • QUALITY ASSURANCE

Pre-Release QA Automation Pyramid



Unit Tests (70% Volume)

JUnit/MockK for Kotlin tariff calculators; Vitest for React components. Fast execution (< 2 minutes).

Integration & Contract (20% Volume)

Pact contract verification testing REST API payloads and simulated OCPP 1.6 WebSocket frames.

End-to-End Automation (10% Volume)

Playwright simulates vendor tariff update; Appium executes driver booking flow on real Android device matrix.

MODULE 04 • RELEASE ENGINEERING

CI/CD Build Cadence: Daily, Release & Hotfix

EVERY NIGHT @ 01:00 AM

1. Daily Build

Compiled automatically from main branch. Runs long regression test suites and end-to-end integration flows.

Target: Internal Staging QA

BI-WEEKLY SPRINT TRAIN

2. Release Build

Freeze branch (e.g. release/v2.4.0). Undergoes penetration testing, stress testing, and Play Store bundle generation.

Target: Production Fleet

UNSCHEDULED EMERGENCY

3. Hotfix Build

Branched directly from production tag to patch critical P0 bugs (e.g. payment signature failure). Bypasses standard queue.

Target: Immediate Patch

MODULE 04 • DISASTER RECOVERY

Pre-Production DB Corruption: Prevention & Fixes

The Failure Scenario

A buggy nightly migration script alters foreign keys incorrectly, corrupting 40,000 reservation records in pre-prod.

Point-in-Time Recovery

Database restored by replaying Write-Ahead Logs (WAL) to 23:59:59, exactly 1 second before the bad migration ran.

Automated Dry-Runs

CI pipeline now spins up disposable Testcontainers DB to execute and roll back migrations before applying to pre-prod.

MODULE 04 • PRODUCTION DEPLOYMENT

Incremental Datacenter Rollout Architecture

PHASE 1 • CANARY

Mumbai DC (5% Traffic)

Deployed to pilot DC. Anycast DNS routes 5% of driver requests. Observe error rates for 45 minutes.

PHASE 2 • REGIONAL

Bengaluru & Chennai (45%)

South region DCs upgraded. Database read-replicas verified for replication lag under active load.

PHASE 3 • FLEET SYNC

Delhi & Hyderabad (100%)

All national DCs synchronized. If any node degrades, traffic fails over to alternate DC in < 5 seconds.

Blast Radius Containment: Deploying incrementally guarantees that a latent memory leak or bad config impacts only 5% of users rather than taking down the entire national grid.

End-to-End CI/CD: Commit to Play Store and Production

A gated pipeline connects GitHub collaboration, backend deployment and Android distribution.

1 Commit & PR
GitHub branch
Review + checks

2 Build
Gradle / React
Reproducible artifacts

3 Verify
Unit • contract
UI • security scans

4 Package
Docker image
Signed Android AAB

5 Deploy
Backend staging
Play internal track

6 Promote
Approval gate
Canary / staged rollout

Quality gates

Fail fast on tests, lint, dependency vulnerabilities, secret scans, migration checks and API contract drift.

Release controls

Require human approval for production and Play Store promotion. Preserve rollback artifacts and deployment metadata.

Android Deployment: AAB to Google Play

A production release path for the UrjaGrid Kotlin consumer application.

Build the Android App Bundle

Generate a signed .aab from a tagged release.

Set versionCode and versionName.

Run lint, unit tests and instrumentation tests.

Keep signing keys outside source control.

Validate the release

Verify manifest, permissions and target SDK.

Test on representative Android API levels.

Check app size, startup and crash-free sessions.

Upload to Play Console internal testing first.

Promote safely

Internal → closed → open testing → production.

Use staged rollout (e.g. 5%, 20%, 50%, 100%).

Monitor crashes, ANRs and key business flows.

Halt rollout / roll forward if health regresses.

Signing & artifact integrity

Use Play App Signing; protect upload key in a managed secret store. Record artifact checksum, build commit SHA and release metadata for traceability.

CI/CD automation

Pipeline builds AAB → signs with protected credentials → runs checks → uploads to Play Console track → waits for approval / promotion gate.

AI Agents in CI/CD: From Reactive Pipeline to Assisted Engineering

Agents can investigate failures and prepare actions; deterministic CI gates and human approvals remain authoritative.

Build & test agent

Reads pipeline output and test failures.

Groups related failures and identifies likely root cause.

Suggests a patch or test improvement.

Creates a PR for developer review.

Security & quality agent

Reviews dependency and static-analysis findings.

Prioritizes exploitable / high-impact issues.

Explains risky changes and missing tests.

Cannot waive mandatory security gates.

Release & incident agent

Summarizes release readiness and change risk.

Checks rollout health metrics.

Recommends pause or rollback when thresholds breach.

Requires authorized approval for production actions.

Example agent loop

Trigger → gather context → reason → call approved tools → inspect result → report / open PR → wait for approval where needed.

Guardrails

Scoped credentials, allowlisted tools, audit trail, sandboxed execution, no secrets in prompts, human approval for deploy / rollback, and bounded retries.

Reference Architecture: AI-Agent-Assisted CI/CD

Illustrative architecture for UrjaGrid. The agent orchestrates work; the pipeline enforces policy.

Event sources

GitHub PR / push

CI failure • alert

›

Agent orchestrator

Task state • plan

LLM reasoning

›

Tool gateway

Scoped APIs

Policy + audit

›

CI/CD runner

Build • test

scan • package

›

Environments

Staging

Play tracks / prod

Context & memory

Repository docs, build logs, test reports, deployment history and service telemetry. Retrieve only relevant, access-controlled context.

Human approval gates

Code owner review, security sign-off and release manager approval before merge, production deployment or rollback.

Feedback & observability

Publish PR comments, run summaries and release health. Feed outcomes back into agent evaluation and prompt / tool improvements.

Trust boundary: agents propose and coordinate; CI policy, signed artifacts, protected environments and authorized humans decide what reaches production.

MODULE 05 · GOVERNANCE

Safe Operating Model for Agentic CI/CD

Use autonomy for investigation and low-risk assistance; keep irreversible production actions controlled.

Autonomous (low risk)

- Summarize logs and test failures
- Suggest owners and likely causes
- Draft release notes
- Open a remediation PR in a sandbox

Approval required

- Merge code
- Rotate production secrets
- Promote AAB to production
- Change infrastructure or database schema

Never delegate blindly

- Expose credentials or customer data
- Disable security / quality gates
- Run unreviewed arbitrary commands
- Approve its own production change

Measure agent effectiveness

Time to triage, actionable suggestion rate, PR acceptance rate, escaped defects, false positives and cost per workflow.

Roll out incrementally

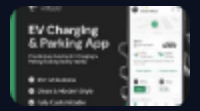
Start in read-only mode → allow draft PRs → enable bounded low-risk actions → expand only after evaluation and audit.

Questions & Architecture Discussion

Connecting React, Kotlin, GitHub collaboration, Android AAB releases, security, performance, CI/CD gates, AI-agent orchestration, and phased deployments.

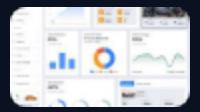
UrjaGrid EV Network Architecture Blueprint

Image Sources



https://images.ui8.net/uploads/plugnpark_1770791133272.png

Source: ui8.net



https://multipurposethemes.com/wp-content/uploads/2025/11/Dashboard-2.png

Source: multipurposethemes.com



https://git-scm.com/book/en/v2/images/perils-of-rebasing-2.png

Source: git-scm.com



https://www.paloaltonetworks.com.br/content/dam/pan/en_US/images/cyberpedia/various-steps-in-the-ci-cd-pipeline.png?imwidth=480

Source: www.paloaltonetworks.com.br